



Informe de Pruebas Unitarias

Proyecto: Innovatech Solutions

Asignatura: DSY1106 · Desarrollo Full Stack

Autor: Najeeb Escobar Pérez

Fecha: Junio 2026

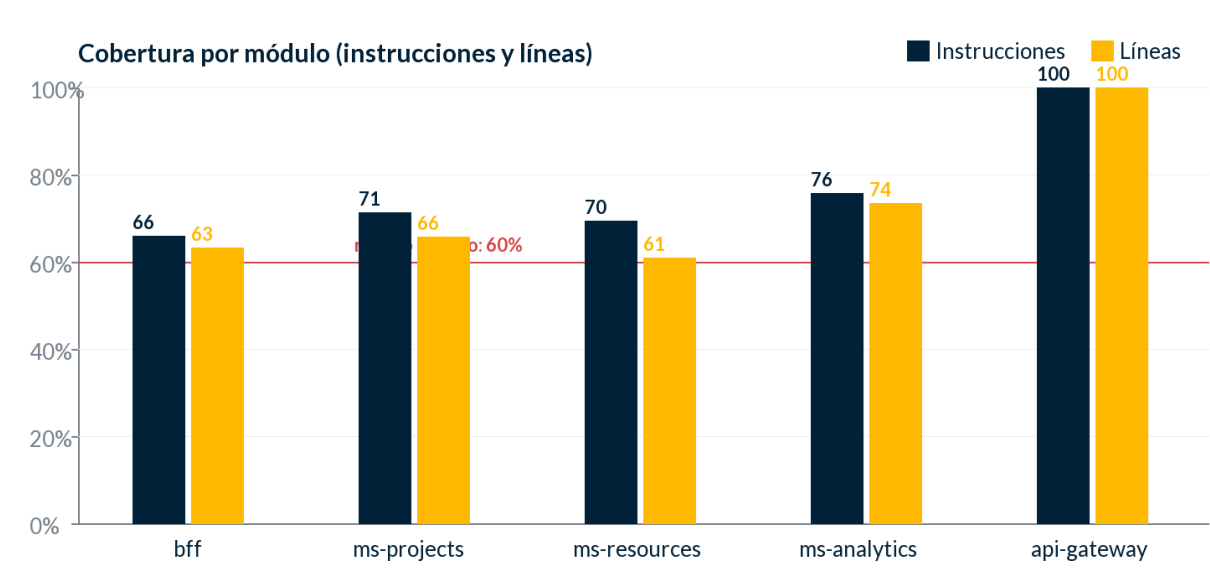
1. Resumen

El backend se prueba con **112 pruebas unitarias** distribuidas en seis módulos Maven, ejecutadas con JUnit 5 y Mockito. En la última corrida **todas pasaron, sin fallos ni errores**. La cobertura se mide con JaCoCo, con una verificación automática (gate) que exige un mínimo de **60% de cobertura de instrucciones por módulo** en cada build; si un módulo baja de ese umbral, mvn verify falla.

Los cuatro módulos con lógica de negocio (bff, ms-projects, ms-resources, ms-analytics) superan el umbral. El api-gateway llega al 100%; eureka-server solo contiene el arranque de Spring (su única clase se excluye de la medición), por eso no aporta cobertura propia.

2. Cobertura por módulo

Módulo	Pruebas	Instrucciones	Líneas	Ramas	Métodos
bff	59	66.1%	63.4%	49.3%	80.5%
ms-projects	15	71.4%	65.9%	68.2%	62.0%
ms-resources	12	69.5%	61.1%	60.0%	58.8%
ms-analytics	23	75.9%	73.5%	87.9%	61.7%
api-gateway	2	100.0%	100.0%	0.0%	100.0%
eureka-server	1	0.0%	0.0%	0.0%	0.0%
Total	112				



La línea roja marca el mínimo exigido (60% de instrucciones). Todos los módulos de negocio quedan por encima.

3. Herramientas y métricas

Frameworks de prueba: JUnit 5 (motor), Mockito (dobles de prueba / mocks) y AssertJ (aserciones legibles). Para validar DTOs se usa el `Validator` de Jakarta.

Cobertura: JaCoCo instrumenta el bytecode durante `mvn verify` y reporta cuatro métricas: instrucciones, líneas, ramas (branches) y métodos. El reporte HTML navegable queda en `target/site/jacoco/index.html` de cada módulo, y el resumen máquina-legible en `jacoco.csv`.

Gate de calidad: el plugin de JaCoCo está configurado con una regla que rompe el build si la cobertura de instrucciones de un módulo cae bajo 60%. Así la cobertura no es un dato opcional: es una condición para integrar.

4. Ejemplos de pruebas

a) Servicio con fallback del Circuit Breaker (ms-analytics). Verifica que, sin snapshots guardados, el cálculo del historial responde "datos no disponibles" en lugar de fallar:

```
@Test
void history_emptySerieReturnsEmpty() {
    when(repo.findAll(any(Pageable.class)))
        .thenReturn(new PageImpl<>(List.of()));

    var resp = svc(true, 4).history(12, null, null);

    assertThat(resp.status()).isEqualTo("datos no disponibles");
}
// resultado: ✓ pasa
```

b) Lectura del JWT en un DTO (bff). El record `AuthenticatedUser` extrae el rol y el email desde el token:

```
@Test
void from_resolvesRoleAndEmail() {
    var auth = new JwtAuthenticationToken(jwt("dev@x.cl", "devuser"),
        List.of(new SimpleGrantedAuthority("ROLE_DEV")));

    var user = AuthenticatedUser.from(auth);

    assertThat(user.role()).isEqualTo(UserRole.DEV);
    assertThat(user.email()).isEqualTo("dev@x.cl");
}
// resultado: ✓ pasa
```

c) Validación declarativa de entrada (bff). El perfil del registro se valida en el DTO; un valor fuera de {DEV, PM, DIR} produce una violación:

```
@Test
void invalidRole_isRejected() {
    var req = new RegisterRequest("Ana", "Diaz", "ana@x.cl",
        "11.111.111-1", "Secret.1", "BOSS");

    assertThat(validator.validate(req))
        .anyMatch(v -> v.getPropertyPath().toString().equals("role"));
}
// resultado: ✓ pasa
```

5. Cómo ejecutar las pruebas y generar los reportes

Desde la raíz del backend, con el JDK configurado:

```
export JAVA_HOME=/opt/homebrew/opt/openjdk
cd backend

mvn test      # ejecuta todas las pruebas unitarias
mvn verify    # pruebas + cobertura JaCoCo + gate de 60% por módulo
```

Tras `mvn verify`, el reporte de cobertura de cada módulo queda en:

```
backend/<módulo>/target/site/jacoco/index.html
```

Ese HTML permite navegar paquete por paquete y clase por clase, viendo qué líneas están cubiertas (verde), parcialmente cubiertas (amarillo) o sin cubrir (rojo).